

University of Westminster

School of Computer Science and Engineering

5COSC022W	Client-Server Architectures - Coursework (2025/26)
Module leader	Hamed Hamzeh
Unit	Coursework (Referral / Deferral)
Weighting:	60%
Qualifying mark	30%
Description	REST API design, development and implementation.
This assignment contributes towards the following Learning Outcomes (LOS):	
Learning Outcomes Covered in this Assignment:	LO1 Gain a thorough understanding of RESTful principles and their application in API design. LO2 Acquire familiarity with the JAX-RS framework as a tool for building RESTful APIs in Java.
Handed Out:	April 2026
Due Date	Tuesday, 30th June 2026, 13:00
Expected deliverables	A public GitHub repo link containing all parts of the project
Method of Submission:	Video demonstration Report included in README.md file in GitHub Electronic submission on Blackboard via a provided link close to the submission time.
Type of Feedback and Due Date:	Written feedback within 15 working days.
BCS CRITERIA MEETING IN THIS ASSIGNMENT	2.1.1 Knowledge and understanding of facts, concepts, principles & theories 2.1.2 Use of such knowledge in modelling and design 2.1.3 Problem solving strategies 2.2.1 Specify, design or construct computer-based systems 2.2.4 Deploy tools effectively 2.3.2 Development of general transferable skills 3.1.1 Deploy systems to meet business goals 4.1.1 Knowledge and understanding of scientific and engineering principles 4.1.3 Knowledge and understanding of computational modelling

Assessment regulations

Refer to section 4 of the "How you study" guide for undergraduate students for a clarification of how you are assessed, penalties and late submissions, what constitutes plagiarism etc.

Penalty for Late Submission

If you submit your coursework late but within 24 hours or one working day of the specified deadline, 10 marks will be deducted from the final mark, as a penalty for late submission, except for work which obtains a mark in the range 40-49%, in which case the mark will be capped at the pass mark (40%). If you submit your coursework more than 24 hours or more than one working day after the specified deadline you will be given a mark of zero for the work in question unless a claim of Mitigating Circumstances has been submitted and accepted as valid.

It is recognised that on occasion, illness or a personal crisis can mean that you fail to submit a piece of work on time. In such cases you must inform the Campus Office in writing on a mitigating circumstances form, giving the reason for your late or non-submission. You must provide relevant documentary evidence with the form. This information will be reported to the relevant Assessment Board that will decide whether the mark of zero shall stand. For more detailed information regarding University Assessment Regulations, please refer to the following website:

<http://www.westminster.ac.uk/study/current-students/resources/academic-regulations>

Learning Goals:

- **Define** key principles of REST architecture.
- **Differentiate** between RESTful and non-RESTful APIs.
- **Recognise** the importance of resource-based interactions.
- **Understand** the role of JAX-RS in Java-based API development.
- **Explore** JAX-RS annotations for resource mapping and HTTP method handling.
- **Implement** basic resource classes using JAX-RS.

Coursework: Developing a JAX-RS RESTful Service (Total 100 Marks)

Module: Client-Server Architectures

Title: "MLOps" Pipeline Management API (Reassessment)

Weight: 60% of final grade

1. Introduction & Scenario

Scenario: You have been appointed as the Lead Backend Architect for an AI research lab that is transitioning its machine learning models into a cloud-native production environment. The lab urgently requires a robust, scalable, and highly available RESTful API to manage their Machine Learning Workspaces and the diverse array of Models (e.g., Deep

Learning classifiers, LLMs, regression algorithms) deployed within them.

This system will be built as a high-performance web service using JAX-RS (Jakarta RESTful Web Services). Your goal is to provide a seamless interface for data scientists and automated MLOps pipelines to monitor training metrics and interact with the deployed AI models.

Objective: This coursework is designed to simulate a real-world development task. It assesses your proficiency in RESTful architectural patterns, the implementation of resource nesting, and the design of a resilient error-handling strategy using JAX-RS. You are expected to demonstrate industry-standard practices, including the use of appropriate HTTP status codes, meaningful JSON responses, and a logical resource hierarchy that reflects the structure of a cloud-based AI platform.

Core "Resource" Data Models:

You will model three primary entities that represent the core pillars of the MLOps infrastructure. Use these POJOs as your foundation, ensuring they are properly encapsulated with getters and setters:

```
1      public class MLWorkspace {
2          private String id;                // Unique identifier, e.g., "WS-
3              VISION-01"
4          private String teamName;          // Human-readable team name, e.g.,
5              "Computer Vision Lab"
6          private int storageQuotaGb;        // Maximum storage allocated for
7              datasets in GB
8          private List<String> modelIds = new ArrayList<>(); // Collection of
9              IDs of models deployed here
10
11         // Constructors, getters, setters...
12     }
13
14     public class MachineLearningModel {
15         private String id;                // Unique identifier, e.g., "MOD
16             -8832"
17         private String framework;          // Category, e.g., "TensorFlow", "
18             PyTorch", "Scikit-Learn"
19         private String status;              // Current state: "TRAINING", "
20             DEPLOYED", or "DEPRECATED"
21         private double latestAccuracy;      // The most recent accuracy score
22             recorded
23         private String workspaceId;         // Foreign key linking to the
24             Workspace hosting the model
25
26         // Constructors, getters, setters...
27     }
28
29     public class EvaluationMetric {
30         private String id;                // Unique metric event ID (UUID
31             recommended)
32         private long timestamp;             // Epoch time (ms) when the metric
33             was captured
34         private double accuracyScore;       // The accuracy/F1 score recorded
35             during the evaluation run
36
37         // Constructors, getters, setters...
38     }
```

2. Coursework Tasks (Step-by-Step)

Part 1: Service Architecture & Setup (10 Marks)

1. Project & Application Configuration (5 Marks):

- Bootstrap a Maven project integrating a JAX-RS implementation (e.g., Jersey) and a lightweight servlet container or embedded server.
- Implement a subclass of `javax.ws.rs.core.Application` and use the `@ApplicationPath("/api/v1")` annotation to establish your API's versioned entry point.
- **Question:** When returning a Java object from a method, it is automatically serialised into JSON. Explain the role of a `MessageBodyWriter` or a JSON provider (like Jackson) in this conversion process.

2. The "Discovery" Endpoint (5 Marks):

- Implement a root "Discovery" endpoint at `GET /api/v1`. This should return a JSON object providing essential API metadata: versioning info, administrative contact details, and a map of primary resource collections (e.g., "workspaces" -> `"/api/v1/workspaces"`).
- **Question:** REST architecture dictates that APIs should be strictly 'stateless'. Define what statelessness means in this context and explain why it makes cloud APIs easier to scale horizontally across multiple servers.

Part 2: Workspace Management (20 Marks)

1. Workspace Resource Implementation (10 Marks):

- Develop a `WorkspaceResource` class to manage the `/api/v1/workspaces` path.
- `GET /`: Provide a comprehensive list of all ML workspaces.
- `POST /`: Enable the creation of new workspaces. Ensure the service returns appropriate feedback upon success.
- `GET /{workspaceId}`: Allow users to fetch detailed metadata for a specific workspace.
- **Question:** Discuss how implementing `HTTP Cache-Control` headers on the `GET workspaces` endpoint could improve performance for the client and reduce unnecessary processing load on the server.

2. Workspace Deletion & Safety Logic (10 Marks):

- Implement `DELETE /{workspaceId}` to allow workspace removal.
- **Business Logic Constraint:** To prevent data orphans, a workspace cannot be deleted if it still has models assigned to it. If a deletion is attempted on a populated workspace, your service must block the request and return a custom error response (as detailed in Part 5).
- **Question:** If a client needs to verify whether a specific workspace exists but wants to save bandwidth by not downloading the entire JSON body, which HTTP method should they use instead of GET? Explain your reasoning.

Part 3: Model Operations & Linking (20 Marks)

1. Model Resource & Integrity (10 Marks):

- Implement `ModelResource` to manage the `/api/v1/models` collection.
- `POST /`: When a new model is registered, your logic must verify that the `workspaceId` specified in the request body actually exists in the system.
- **Question:** When creating a new Model via a `POST` request, it is considered best practice for the server to generate the unique `id` (e.g., using `UUID.randomUUID()`) rather than allowing the client to pass an `id` in their JSON payload. Discuss the security and data integrity reasons behind this architectural choice.

2. Filtered Retrieval & Search (10 Marks):

- Enhance `GET /api/v1/models` to support an optional query parameter named `status` (e.g., `GET /api/v1/models?status=DEPLOYED`). If provided, the response must filter the list to only include matching ML models.
- **Question:** If a user attempts to search for a framework containing spaces or special characters (e.g., `?framework=Scikit Learn & Tools`), how must the client modify the URL, and why is this encoding necessary?

Part 4: Deep Nesting with Sub-Resources (20 Marks)

A key requirement is to maintain a historical log of evaluation metrics for every model to track concept drift and performance over time.

1. The Sub-Resource Locator Pattern (10 Marks):

- In your `ModelResource` class, implement a sub-resource locator method for the path `{modelId}/metrics`. This method should return an instance of a dedicated `EvaluationMetricResource` class.
- **Question:** You can place annotations like `@Produces(MediaType.APPLICATION_JSON)` at either the class level or the individual method level. What is the benefit of class-level placement, and how does method-level overriding work?

2. Historical Data Management (10 Marks):

- Within `EvaluationMetricResource`, implement `GET /` to fetch history and `POST /` to append new evaluation metrics for that specific model context.
- **Side Effect:** A successful `POST` to a metric must trigger an update to the `latestAccuracy` field on the corresponding parent `MachineLearningModel` object to ensure data consistency across the API.

Part 5: Advanced Error Handling, Exception Mapping & Logging (30 Marks)

You must ensure the API is “leak-proof”—it should never return a raw Java stack trace or a default server error page. Furthermore, the API requires observability through proper request and response logging. Implement specific Exception Mappers and Filters for these scenarios.

1. Resource Conflict (409) (5 Marks):

- **Scenario:** Attempting to delete an ML Workspace that still has Models assigned to it.

- **Task:** Create a custom `WorkspaceNotEmptyException`. Implement an Exception Mapper that returns an HTTP 409 Conflict with a JSON body explaining the constraint.

2. Dependency Validation (422 or 400) (10 Marks):

- **Scenario:** A client attempts to POST a new Model with a `workspaceId` that does not exist.
- **Task:** Create a `LinkedWorkspaceNotFoundException`. Use an Exception Mapper to return an appropriate error status (HTTP 422 Unprocessable Entity or 400 Bad Request).
- **Question:** HTTP status codes are categorised into classes (e.g., 2xx, 4xx, 5xx). Explain fundamentally why a validation failure caused by the user providing a non-existent `workspaceId` must return a 4xx code rather than a 5xx code.

3. State Constraint (403 Forbidden) (5 Marks):

- **Scenario:** A model currently marked with the status "DEPRECATED" cannot accept new evaluation metrics as it is no longer being monitored.
- **Task:** Create a `ModelDeprecatedException`. Map this to an HTTP 403 Forbidden status when a POST metric is attempted on a deprecated model.

4. The Global Safety Net (500) (5 Marks):

- **Task:** Implement a "catch-all" `ExceptionHandler<Throwable>`. This mapper must intercept any unexpected runtime errors (e.g., `NullPointerException`, `IndexOutOfBoundsException`) and return a generic HTTP 500 Internal Server Error.
- **Question:** If an operation throws a specific custom exception (e.g., `LinkedWorkspaceNotFoundException`) and you also have a global `ExceptionHandler<Throwable>`, how does the JAX-RS runtime determine which mapper to execute?

5. API Request & Response Logging Filters (5 Marks):

- **Task:** Implement API observability by creating a custom filter class that implements both `ContainerRequestFilter` and `ContainerResponseFilter`. Use `java.util.logging.Logger` to log the HTTP method, URI, and final status code for incoming and outgoing requests.
- **Question:** In your filter, you interact with `ContainerRequestContext` and `ContainerResponseContext`. List two pieces of crucial HTTP metadata (e.g., headers, URIs) you can extract from these contexts that are highly valuable for debugging server issues.

3. Submission & Marking Breakdown

Professional Practice (Mandatory Prerequisites)

GitHub Hosting: The project must be hosted in a public GitHub repository. Your repository must contain a `README.md` that provides:

- An overview of your API design.
- Explicit, step-by-step instructions on how to build the project and launch the server.
- At least five sample curl commands demonstrating successful interactions with different parts of the API.
- Answers to questions for each part of the coursework.

Video Demonstration: A maximum of 10-minute video demonstration of Postman tests. The video must be uploaded directly to the Blackboard submission link. There is no need to show the code; however, the development environment must be shown during the video demonstration. Please note that the video demonstration is mandatory for the assessment. In the video demonstration, you **MUST** be present and speak clearly.

Report: The report must be prepared in a `README.md` file in the GitHub repo, which must only include the answers to the questions in each part. There is no need to include test cases and an introduction.

Evaluation Criteria

- Part 1 (Setup & Discovery): 10 Marks
- Part 2 (Workspace Management): 20 Marks
- Part 3 (Models & Filtering): 20 Marks
- Part 4 (Sub-Resources): 20 Marks
- Part 5 (Error Handling & Logging): 30 Marks
- **Conceptual Report:** This document must only contain the written answers to the questions found in each section of your report. Keep in mind that your report's overall mark is influenced by your performance on each task, with 20% of the total score allocated to answering specific questions. For instance, in task 3.1, you can earn 2 marks out of 10 by including a proper response to the question posed. Be sure to address these questions thoroughly to maximise your scores! Please note that the report must be organised and written in the `README.md` file on GitHub.
- **Video Demonstration:** Please keep in mind that each coursework task requires a recorded video demonstration to showcase your work. This video demonstration is significant, as it accounts for 30% of the total score for each task. For instance, in Task 3.1, the video demonstration will contribute 3 marks out of the total 10 marks available for that task. Make sure to put effort into your video presentations, as they are an important part of your overall assessment. Please note that you should be clearly present during the video presentation. Therefore, it is critical to ensure that your camera and microphone are working properly. Failing to submit a recorded video presentation will result in a 30% loss of marks on each task that requires video recording. Please note that the Video Demonstration link/file must be included in the Blackboard submission link.

- **Coding:** Please note that the allocated mark for coding for each section is 50%, which means that if a section has 10 marks, 5 marks are allocated to coding, 3 marks are allocated to video demonstration, and 2 marks are allocated to answering questions in the report.
- **Technology Stack:** You must only use JAX-RS. Using other technologies like Spring Boot is not allowed, which means that you will receive an immediate ZERO mark for the whole coursework.
- **NO ZIP FILE:** PLEASE NOTE THAT, SUBMITTING ANY ZIP FILE INSTEAD OF HOSTING THE APP ON GITHUB WILL RESULT IN A ZERO MARK FOR THE WHOLE COURSEWORK.
- **Database:** You are not allowed to use any database technology like SQL Server. Otherwise, you will receive a ZERO mark for the coursework. You must only use data structures like HashMap or ArrayList.